04a Schleifen

04a Schleifen

00 Übersicht · Zurück: 04 if-Anweisung · Weiter: 04b Switch

Schleifen sind Kontrollstrukturen zur **wiederholten Ausführung** von Anweisungen. Die Anzahl der Durchläufe wird über eine **Bedingung** gesteuert (03a Operatoren in Bedingungen).

Schleife	Prüfung	Typ
`while`	**vor** dem Durchlauf	**abweisende** Schleife
`do-while`	**nach** dem Durchlauf (min. 1×)	**annehmende** Schleife
`for`	vor dem Durchlauf, mit **Schleifenzähler**	bekannter Wertebereich

while-Schleife

```
while (Bedingung) {
    Anweisungsliste
    Anpassung der Bedingung
}
…
```

Ablauf

1. Prüfe die Bedingung.
2. `true` → führe den `while`-Block aus, gehe zurück zu 1.
3. `false` → verlasse die Schleife, fahre im Programm fort.

> **Endlosschleife**
> Die Bedingung muss im Schleifenkörper **angepasst** werden, sonst droht eine Endlosschleife.

Übung: alle Vielfachen von drei zwischen 1 und `boundary` ausgeben (Lösung der Folie):

```java
private static final int DIV = 3;

public static void main(String[] args) {
    int boundary = Integer.parseInt(args[0]);
    // ggf. Prüfung von boundary.
    int counter = 1;
    while (counter <= boundary) {
        if (counter % DIV == 0) {
            System.out.println(counter);
        }
        counter++;
```

```
    }
}
```

do-while-Schleife

Variante der `while`-Schleife: die Abbruchbedingung wird **erst nach dem ersten Durchlauf** geprüft.

```
do {
    Anweisungsliste
    Anpassung der Bedingung
} while (Bedingung)
```
…

Ablauf: 1. Gehe in den `do`-Block → 2. führe Anweisungen aus → 3. prüfe Bedingung: `true` → zurück zu 1; `false` → Schleife verlassen.

> **Der `do`-Block läuft** mindestens einmal**.**
> `do-while` = **annehmend**, `while` = **abweisend**.

Übung: alle Zahlen von `boundary` abwärts bis zum größten Vielfachen einer Zahl `value` (mit `value <= boundary`) ausgeben (Lösung der Folie):

```
public static void main(String[] args) {
    int boundary = Integer.parseInt(args[0]);
    int value = Integer.parseInt(args[1]);
    do {
        System.out.println(boundary);
        boundary = boundary - 1;
    } while (boundary % value != 0);
}
```

Beispiel `boundary = 10`, `value = 3`: gibt `10` aus, danach ist `boundary = 9` (Vielfaches von 3) → Abbruch. Das Vielfache selbst (9) wird in dieser Folienlösung **nicht** mehr ausgegeben; das ist ein Effekt der `do-while`-Reihenfolge (erst ausgeben, dann prüfen).

for-Schleife

Wird verwendet, wenn ein **bekannter Wertebereich** mit einem **Schleifenzähler** durchlaufen werden soll (z.B. Zahlen 1 bis 100, Array bestimmter Länge).

```
for (Initialisierung sz; Bedingung; sz weiterzählen) {
    Anweisungsliste
}
```
…

Ablauf
1. Initialisiere den Schleifenzähler.
2. Prüfe die Bedingung: `true` → Anweisungsliste ausführen, Zähler anpassen, zurück zu 2; `false` → Schleife verlassen.

Beispiel/Übungen (Lösungen der Folien):

```
// Übung 1: Vielfache von 3 bis boundary
int boundary = Integer.parseInt(args[0]);
for (int i = 0; i < boundary; i = i + 3) {
```

```
    System.out.println(i);
}

// Übung 2: Quersumme einer positiven Ganzzahl
int value = Integer.parseInt(args[0]);
int sum = 0;
for (int i = value; i > 0; i = i / 10) {
    int next = i % 10;      // letzte Ziffer
    sum = sum + next;
}
System.out.println("Die Quersumme von " + value + " ist: " + sum);
```

Anmerkung zur Folienlösung (Übung 1)
Die Lösung startet bei `i = 0` und prüft `i < boundary`: sie gibt also auch die `0` aus und lässt `boundary` selbst aus, wenn es ein Vielfaches von 3 ist. Für „zwischen 1 und boundary“ wäre `for (int i = 3; i <= boundary; i += 3)` genauer.

Quersumme
`i % 10` liefert die letzte Ziffer, `i / 10` (ganzzahlige Division, 02e Arithmetische Operatoren) schneidet sie ab.

Operatoren zur Schleifenkontrolle

Operatorzuweisung (mit allen arithmetischen Operatoren möglich):

```
int i = 1;
i += 2;   // entspricht i = i + 2;
i -= 2;   // entspricht i = i - 2;
```

Inkrement `++` (nur Hochzählen in Einserschritten) und **Dekrement** `--` (nur Herunterzählen in Einserschritten):

```
int i = 1;  i++;   // i = i + 1
int i = 10; i--;   // i = i - 1
```

Präfix- und Postfix-Schreibweise

	Präfix	Postfix
Inkrement	`++variable`	`variable++`
Dekrement	`--variable`	`variable--`
Wirkung	**erst** ändern, **dann** Wert liefern	**erst** Wert liefern, **dann** ändern

Aufgabe: Ausgabe der vier Beispiele von der Folie → 99 Übungsaufgaben

foreach

In den Themen der Vorlesung angekündigt (`while`, `do-while`, `for`, `foreach`), auf den Folien aber **nicht ausgeführt**. Hier fehlt also noch Stoff (Platz für eigene Notizen, Bezug: 02f Arrays).